CIS 3300 · INHERITANCE AND POLYMORPHISM, PART 4 OF 4

Multiple Inheritance

The diamond problem, and how Java uses interfaces to get most of the benefit with little of the risk

---

You will often hear that "Java does not support multiple inheritance." That sentence is true about classes, but it is not the whole story. This last set of notes explains what multiple inheritance is, why it causes trouble, and exactly how much of it Java allows, by separating the three things a class can inherit: state, behavior, and type. It finishes the payroll example with an interface that employees share with people who are not employees. For reading, see Core Java, Volume I, Chapter 5, Section 5.1, and Chapter 6, Section 6.1.

A note before you begin. These are my notes from over the years, cleaned up so they are readable. They are not a study guide, and they are not gospel: not everything here appears in a lecture, and not every topic has notes yet. I write from memory and revise every semester. I also use AI to help convert documents to LaTeX, check for issues, and add explanations. I do not trust it completely, so I am still editing, and a typo or imprecise statement can survive longer than it should. Java also changes over time, though the basics we cover are resilient. If you find something that looks wrong, contradicts the book, or does not behave the way I claim when you run it, tell me. You will be doing the other students a favor, and you will understand the material better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. These notes sometimes go beyond the book, but they do not replace the assigned chapters or the recorded lectures, and neither of those replaces the notes. Read the chapter, watch the lecture, and study the notes.

1. What Multiple Inheritance Is

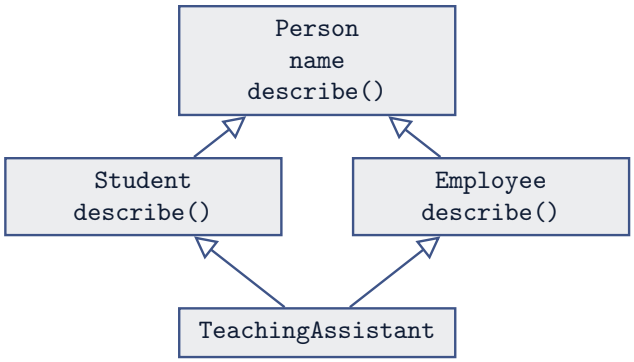
Multiple inheritance means a class has more than one direct superclass. Some languages, such as C++ and Python, allow it. If Java allowed it, you could write something like this, which does not compile:

```
public class TeachingAssistant extends Student, Employee { }   // not legal Java
```

The appeal is obvious: a teaching assistant really is both a student and an employee, and it would be convenient to inherit everything from both. The trouble starts when both superclasses have something in common.

2. The Diamond Problem

Suppose `Person` has a `name` field, a constructor, and a `describe()` method. `Student` and `Employee` both extend `Person`, and each overrides `describe()`. Now `TeachingAssistant` extends both. Drawn as a diagram, the four classes form a diamond (Figure 1).



Behavior: which `describe()` does a `TeachingAssistant` inherit?

State: one copy of `name`, or two?

Construction: does `Person`'s constructor run once or twice?

Figure 1: The diamond problem. Each arrow points from a subclass to a superclass. `TeachingAssistant` reaches `Person` by two paths, which raises three questions that have no single obvious answer.

- **Behavior.** `Student.describe()` and `Employee.describe()` have the same signature, and a class can have only one method with a given signature. Which one wins?
- **State.** Does a teaching assistant hold one `name` or two? If two, which one does `getName()` return? If one, which path's constructor sets it?
- **Construction.** Part 3 showed that every constructor chains up to its superclass. With two paths up, `Person`'s constructor would be reached twice, possibly with different arguments.

Languages that allow multiple inheritance of classes need extra rules for every one of these questions, and those rules are a common source of bugs. Java's designers chose to avoid the problem for classes entirely.

3. State, Behavior, and Type

The sentence "Java does not support multiple inheritance" becomes precise once you split it into the three things a class inherits.

Table 1: How much multiple inheritance Java allows.

Multiple inheritance of	Allowed?	How
State	No	Only classes have instance fields, and a class extends only one class. Interfaces can declare constants, but no instance fields.
Type	Yes	A class can implement any number of interfaces, and its objects are instances of every one of those types.
Behavior	Limited	Since Java 8, interfaces can provide `default` methods, so a class can inherit method bodies from several interfaces. Java has firm rules for conflicts (Section 6).

So the accurate version of the sentence is: *Java does not allow multiple inheritance of state. It allows multiple inheritance of type through interfaces, and a limited, rule-governed form of multiple inheritance of behavior through default methods.*

Who can extend or implement what

Table 2: The legal combinations of extends and implements.

A...	Can	Cannot
Class	`extends` exactly one class (`Object` if none is named), and `implements` any number of interfaces	Extend an interface, implement a class, or extend two classes
Interface	`extends` any number of interfaces	Extend a class, or implement anything

The order in a class header is fixed: `extends` first, then `implements`.

```
public class Manager extends SalariedEmployee implements Mentor, Trainer { ... }
public interface Leader extends Mentor, Trainer { ... }
```

4. Multiple Inheritance of Type: Payable

The company also pays outside contractors. A contractor sends a monthly invoice, but is not an employee: no employee ID, no salary, nothing to inherit from `Employee`. Still, payroll wants one list of everyone to pay. An interface gives employees and contractors a shared type without forcing them into one hierarchy.

```
public interface Payable {
    String getName();
    double getMonthlyPay();

    default String paymentLine() {
        return String.format("%-10s %10.2f", getName(), getMonthlyPay());
    }
}
```

`Employee` only has to add `implements Payable`. It already has a `getName()` method, and its abstract `getMonthlyPay()` matches the interface, so none of its subclasses change. A superclass method can satisfy an interface's requirement on behalf of every subclass.

```
public abstract class Employee implements Payable { ... }   // nothing else changes

public class Contractor implements Payable {
    private final String name;
    private final String company;
    private double monthlyInvoice;

    public Contractor(String name, String company, double monthlyInvoice) {
        this.name = name;
        this.company = company;
        this.monthlyInvoice = monthlyInvoice;
    }
```

```
    @Override public String getName() { return name; }
    @Override public double getMonthlyPay() { return monthlyInvoice; }
    public String getCompany() { return company; }
}
```

Now one list, and one method, can handle both:

```
public static void printPayments(ArrayList<Payable> payees) {
    double total = 0;
    for (Payable p : payees) {
        System.out.println("  " + p.paymentLine());
        total += p.getMonthlyPay();
    }
    System.out.printf("  %-10s %10.2f%n", "TOTAL", total);
}
```

```
  Ada           5000.00
  Linus         5250.00
  Grace         8500.00
  Alan         17000.00
  Margaret      7200.00
  TOTAL        42950.00
```

Everything from Part 2 still applies, with an interface as the declared type. The declared type `Payable` allows only `getName`, `getMonthlyPay`, and `paymentLine`. To reach `getCompany`, you test and cast:

```
for (Payable p : payees) {
    if (p instanceof Contractor c) {
        System.out.println(c.getName() + " works for " + c.getCompany());
    }
}
```

One object, many types

By the end of the unit, an `Executive` object passes eight is-a tests. Five come from its chain of classes: `Executive`, `Manager`, `SalariedEmployee`, `Employee`, and `Object`. Three come from interfaces: `Payable`, plus `Mentor` and `Trainer` from the next section. It inherited state from one chain of classes, but it has many types. That is multiple inheritance of type.

5. What a Class Actually Gets From an Interface

Interfaces are not classes, and nothing in them is an object, so "inheriting from an interface" works a little differently from inheriting from a class. This table pulls together the interface features from the earlier unit and states what an implementing class receives.

Table 3: Interface members and what an implementing class receives.

Interface member	What the implementing class gets
Abstract method	An obligation. A concrete class must provide a body. An abstract class may leave it to its subclasses.
`default` **method**	A ready-made body that is inherited, and that the class may override
Constant	Inherited: the class can use it by its simple name. Constants are always `public static final`, so there is one shared copy, not per-object state.
`static` **method**	Not inherited. Call it through the interface name, as in `Payable.someHelper()`.
`private` **method**	Not inherited. It exists only to help the interface's own default methods.
Constructor or instance field	Interfaces have neither, which is exactly why they avoid the state and construction parts of the diamond problem

A closer look at constants

A fair question about the constant row: if a constant is `static`, and an interface can never be instantiated, is it really inherited? Yes, but in a narrower sense than an instance field. The listing below is a cut-down `Payable` and `Contractor` with a constant added. The Example 5 code has no constant, so this listing is for illustration only.

```
interface Payable {
    double TAX_RATE = 0.20;          // public static final, even though it does not
    say so
    double getMonthlyPay();
}

class Contractor implements Payable {
    private double monthlyInvoice;

    public double getMonthlyPay() { return monthlyInvoice; }

    public double getTax() {
        return getMonthlyPay() * TAX_RATE;   // simple name: TAX_RATE is inherited
    }
}
```

The Java Language Specification says a class inherits the accessible fields of its superclass and its superinterfaces, and that includes interface constants. That is why `TAX_RATE` works by its simple name inside `Contractor`, and why `Contractor.TAX_RATE` also compiles. What the class inherits is the *name*, not a copy of the value. Because the constant is `static`, it belongs to the interface itself: there is exactly one `TAX_RATE`, it exists before any `Contractor` is created, and it would exist even if no class ever implemented `Payable`. A `static` field in a superclass works the same way. Only instance fields give each object its own copy.

Notice the difference from the table's `static` method row. A `static` field declared in an interface is inherited, but a `static` method declared in an interface is not, so `Payable.someHelper()` must always be called through the interface name.

Three details follow from this.

- **Two interfaces, one name.** If a class implements two interfaces that both declare `TAX_RATE`, the simple name is ambiguous and does not compile where it is used. Writing `Payable.TAX_RATE` fixes it. This is the diamond problem for fields, and it stays harmless because the value is final and shared. That is why we say interfaces give multiple inheritance of type and behavior, but not of state.
- **Hiding, not overriding.** A class may declare its own field named `TAX_RATE`. That field *hides* the interface constant. Fields never take part in dynamic binding, so which one you get depends on the declared type, not the actual object.
- **Compile-time constants are copied.** When a constant's value is known at compile time, as `0.20` is, the compiler copies the value into each class that uses it. If you change the interface and recompile only the interface, classes compiled earlier keep the old value until they are recompiled too.

Do not implement an interface just for its constants. Implementing an interface only to use its constants by their short names is a known poor design, called the *constant interface* anti-pattern in Joshua Bloch's *Effective Java*. It makes the class a `Payable` in name only, and every subclass inherits that false type. If you only want short names, use a `static import`, or put the constants in a `final` class with a `private` constructor.

6. The Diamond Returns: Conflicting Default Methods

Default methods brought back one corner of the diamond: two interfaces can each supply a body for the same method. Here are two interfaces, each with a default `supportDescription`:

```
public interface Mentor {
    default String supportDescription() {
        return "meets with a mentee every week";
    }
}

public interface Trainer {
    default String supportDescription() {
        return "runs a training session every month";
    }
}
```

If `Manager` implements both and does nothing else, Java refuses to guess:

```
error: types Mentor and Trainer are incompatible;
  class Manager inherits unrelated defaults for supportDescription() from types Mentor
    and Trainer
```

The fix is to override the method. Inside the override, a call written in the special form

`Mentor.super.supportDescription()` runs that one interface's default body, so you can pick one version, combine them, or write something new:

```
public class Manager extends SalariedEmployee implements Mentor, Trainer {
    // ... fields, constructor, and methods from Part 3 ...

    @Override
    public String supportDescription() {
        return Mentor.super.supportDescription() + " and "
                + Trainer.super.supportDescription();
    }
}
```

```
Grace meets with a mentee every week and runs a training session every month
```

`Executive` extends `Manager`, so it inherits this override and needs no changes.

The three rules

When a class could inherit more than one version of a method, Java applies these rules in order.

1. **Classes win.** A method from a superclass beats any interface default with the same signature. If `Person` has a concrete `greet()` and `Greeter` has a default `greet()`, then `class Student extends Person implements Greeter` uses `Person`'s version without writing anything.
2. **The more specific interface wins.** If `FriendlyGreeter extends Greeter` and overrides its default, a class implementing both uses `FriendlyGreeter`'s version.
3. **Otherwise, you must choose.** If neither rule settles it, as with `Mentor` and `Trainer`, the class must override the method, and it may call `X.super.method()` to reuse either version.

Rule 1 explains a restriction you might otherwise find odd: an interface cannot declare a default `toString`, `equals`, or `hashCode`. Every class already inherits those from `Object`, and classes win, so such a default could never run. The compiler says so:

```
error: default method toString in interface Payable overrides a member of java.lang.
    Object
```

7. Choosing Among the Tools

Abstract class or interface?

Both can declare abstract methods and both define a type, so the choice comes down to the differences in this table.

Table 4: Abstract classes compared with interfaces.

Question	Abstract class	Interface
How many can a class use?	One	Any number
Instance fields?	Yes	No; constants only
Constructors?	Yes	No
Method bodies?	Yes	`default`, `static`, and `private` methods only
Relationship it expresses	"is a kind of": shared identity and shared data	"can do": a capability or role
Payroll example	`Employee`	`Payable`, `Mentor`, `Trainer`

A useful habit: if unrelated classes need to be treated alike (employees and contractors), reach for an interface. If closely related classes share real data and code (every employee has an ID and a name), reach for an abstract class. Many designs use both, exactly as the payroll example does.

Sometimes the answer is has-a

Not every "is both" needs inheritance. A teaching assistant could be modeled as a class that *has* a `Student` record and *has* an `Employee` record, the aggregation relationship from the earlier unit, and implements whatever interfaces it needs. You may see this advice summarized as "favor composition over inheritance." Inheritance is powerful, but it ties a subclass tightly to its superclass. When the is-a sentence feels forced, that is usually a sign to use has-a instead.

8. Summary

Table 5: Summary of the ideas in these notes.

Idea	Key point
Multiple inheritance	A class with more than one direct superclass; not allowed for Java classes
Diamond problem	Two inheritance paths to one ancestor make behavior, state, and construction ambiguous
Multiple inheritance of type	Allowed: a class implements any number of interfaces
Multiple inheritance of behavior	Limited: inherited default methods, with conflict rules
Conflict rules	Classes win; the more specific interface wins; otherwise override and use `X.super.m()`
Interfaces extending interfaces	An interface may extend any number of interfaces
Abstract class or interface	Shared data and identity, or a shared capability
Composition	Has-a is often a better fit than a forced is-a

9. Try It

1. Remove the `supportDescription` override from `Manager` and compile. Then make `HourlyEmployee` implement `Trainer` only. Why does `HourlyEmployee` compile without an override when `Manager` did not?
2. Write an interface `Leader extends Mentor, Trainer`. What must `Leader` do before it compiles? Then make `Executive` implement `Leader`. Which rule decides which `supportDescription` an `Executive` uses now?
3. Add an `Invoice` class that implements `Payable` (a vendor name and an amount) and put an `Invoice` into the payment list. What did you have to change in `printPayments`?